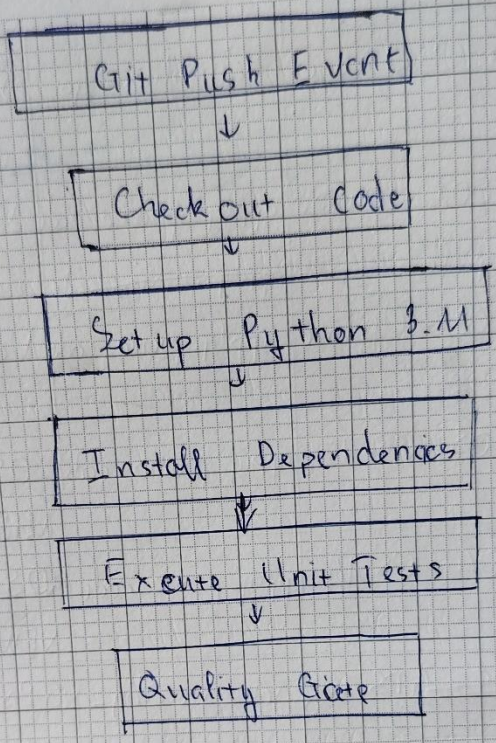
2312756 - Nguyễn Hưng Thịnh - 16/10/2026 - Th4



Khi lập trình viên đẩy mã nguồn lên Git Hub, Github actions tự động tải mã nguồn bằng flake8 và Coverage. Nếu độ bao phủ đạt ít nhất 80% và các bước kiểm tra thành công thì CI báo Pass, ngược lại sẽ báo Fail

Tên Test case	Hàm được gọi	Đối tượng Mocking và giá trị	Kết quả kỳ vọng
TC01 - Online Hợp lệ	apply_promo_code ~~apply_promo_code~~	Network = online mã GIAM10	100.000 → 90.000
TC02 - Offline	apply_promo_code	Network = offline	Ném CorrectionExcep
TC03 - OS 15	apply_promo_code	OS = 15, NEW_OS_ONLY	100.000 = 80.000
TC04 - OS 14	apply_promo_code	OS = 14 NEW_OS_ONLY	Ném ValueError
TC05 - GPS trong vùng	get_nearby_restaurants	GPS = (10.0, 105.0)	Trả về 2 nhà hàng
TC06 - GPS ngoài vùng	get_nearby_restaurants	GPS = (20.0, 110.0)	Trả về danh sách rỗng
TC07 - GPS mất tín hiệu	get_nearby_restaurants	GPS ném GPSSignalLostExcep	Trả về thông báo không xác định được vị trí

Nhận xét: Mocking giúp giả lập các trạng thái mạng và GPS mà không cần sử dụng thiết bị thật. Nhờ đó, các trường hợp lỗi có thể được kiểm thử dễ dàng, stable.

TTBOOK

Câu 2.1: ~~Tại sao~~ Linter cần được chạy trước Unit Test để phát hiện các lỗi sớm, những vấn đề cơ bản trong mã nguồn. Nếu chương trình có lỗi nghiêm trọng, hệ thống CI sẽ dừng mà không thực hiện các bước kiểm thử tiếp theo

Cách tổ chức này tuân theo nguyên tắc Fail-Fast, giúp tiết kiệm thời gian, tài nguyên máy chủ và tăng tốc độ phản hồi của lập trình viên

Câu 2.2: Tham số --fail-under = 80 yêu cầu độ phủ mã nguồn đạt tối thiểu 80%.

Nếu coverage ~~dưới~~ dưới 80%, lệnh trả về Exit Code ~~0~~ ≠ 0 thông thường là 2. GitHub Actions nhận biết mã thoát này và đánh dấu bước kiểm tra là Failed, khiến công việc CI không đạt yêu cầu.

Nếu coverage đạt từ 80% trở lên, lệnh trả về Exit Code 0, nghĩa là thực hiện thành công.

Đây là cơ chế Quality Gate, giúp ngăn mã nguồn chưa đạt tiêu chuẩn chất lượng được tích hợp vào nhánh chính.

Câu 2.3 -Tham số -m (Missing) hiển thị các số dòng mã nguồn chưa được kiểm thử bao phủ.
-Thông tin này giúp các kĩ sư xác định những đoạn chương trình chưa được thực thi trong quá trình kiểm thử, từ đó bổ sung các T/Case còn thiếu
- Nhờ vậy, lập trình viên có thể nâng cao độ bao phủ mã nguồn, giảm nguy cơ bỏ sót lỗi và cải thiện chất lượng phần mềm

Câu 3.1 ~~Flacking~~ Flaky Test: Bài kiểm thử lúc Pass, lúc Fail dù mã nguồn không đổi. Ví dụ: GPS mất tín hiệu ngẫu nhiên
Lỗi môi trường: Kiểm thử thất bại do khác phiên bản thư viện, Py thon hoặc cấu hình hệ thống

Câu 3.2: Version Pinning giúp cố định phiên bản thư viện đảm bảo môi trường kiểm thử đồng nhất tránh lỗi do cập nhật thư viện giúp CI hoạt động ổn định.

Câu 33: Quy trình 6 bước nâng cấp thư viện

1. Tạo nhanh mới từ main
2. Cập nhật phiên bản thư viện
3. Chạy thiên thử cục bộ
4. Commit và Push lên GitHub
5. Tạo Pull Request là chạy CI
6. Review, kiểm tra PASS rồi ~~mege~~ merge vào main.